

Project Documentation: MCQ & Video Generator AI

Project Documentation: MCQ & Video Generator AI

This project is a powerful FastAPI-based tool that leverages AI to generate educational content (Multiple Choice Questions and Videos) from documents in PDF and Word formats.

Table of Contents

1. [Core Features](#core-features)
2. [Libraries and Technologies](#libraries-and-technologies)
3. [Architecture and Implementation](#architecture-and-implementation)
4. [Step-by-Step for Each Feature](#step-by-step-for-each-feature)

Core Features

The project provides four primary functionalities:

1. **MCQ Generation**: Automatically creates set of questions from uploaded documents.
2. **Initial Video Generation**: Creates a structured educational video with slides, narration, and background images.
3. **Video Regeneration from Script**: Allows users to provide a modified script (.txt) to re-generate the video.
4. **Video Regeneration from Transcription**: Synchronizes a modified transcription with existing slides to update the video.

Libraries and Technologies

- **FastAPI**: A modern, high-performance web framework for building APIs with Python.
- **Uvicorn**: A lightning-fast ASGI server implementation for running the FastAPI application.
- **LangChain**: A framework for developing applications powered by large language models (LLMs).
- **Groq API**: High-speed inference for LLMs (like Llama-3) and speech-to-text (Whisper).
- **gTTS (Google Text-to-Speech)**: Converts narrated text into high-quality MP3 audio files.
- **MoviePy**: A library for video editing, used for assembling slide images and audio clips into a final

Project Documentation: MCQ & Video Generator AI

MP4.

- **Pillow (PIL)**: Used to dynamically create slide images with titles, bullets, and branding (logo).
- **PyPDF & python-docx**: Used for robust text extraction from PDF and Word documents respectively.

Architecture and Implementation

The project follows a modular structure:

- **api.py**: Contains the FastAPI endpoints and the core video generation orchestrator (`_process_video_generation``).
- **src/utlis.py**: Houses utility functions for document processing, LLM prompting, image generation, and audio transcription.
- **src/config.py**: Centralized configuration and AI prompt templates.

Step-by-Step for Each Feature

1. Generate MCQs (`/generate-mcqs``)

- **Step 1**: The user uploads a PDF or DOCX file.
- **Step 2**: The `extract_text_from_file`` utility extracts the text and truncates it to the allowed length.
- **Step 3**: A prompt is constructed asking the LLM (Groq) to generate a specific number of MCQs based on the content.
- **Step 4**: The `parse_mcqs`` function processes the LLM's text output into a structured JSON response.

2. Generate Video (`/generate-video``)

- **Step 1**: Text is extracted from the uploaded document.
- **Step 2**: The LLM generates a structured JSON object containing 8-12 slides, each with a Title, Bullet Points, and a Narrator Script.
- **Step 3**: For each slide:
- **Audio**: `gTTS`` generates an MP3 for the script.
- **Image**: `Pillow`` creates a 1280x720 PNG with the slide title, bullets, and logo.

Project Documentation: MCQ & Video Generator AI

- **Clip**: `MoviePy` combines the image and audio, matching the duration.
- **Step 4**: All slide clips are concatenated into a single MP4 video.
- **Step 5**: The video's audio is transcribed back to text using `Groq Whisper` for reference.
- **Step 6**: The MP4, the script text, and the transcription text are bundled into a ZIP file for download.

3. Regenerate Video from Script (`/regenerate-video`)

- **Step 1**: The user uploads a previously generated `slides_content.txt` (the script).
- **Step 2**: If an optional modification prompt or source document is provided, the LLM refines the script.
- **Step 3**: The `parse_script_txt` function converts the modified text back into a slide data structure.
- **Step 4**: The system re-executes the image and audio generation process to create a new video based on the updated script.

4. Regenerate Video from Transcription (`/regenerate-from-transcription`)

- **Step 1**: The user uploads both a `transcription.txt` (which they may have edited) and the original `slides_content.txt`.
- **Step 2**: The `parse_transcription_txt` utility extracts the spoken text for each slide index.
- **Step 3**: The `merge_transcription_with_slides` function replaces the narrator scripts in the slides data with the text from the transcription.
- **Step 4**: An optional user prompt can further modify the merged content via LLM.
- **Step 5**: The system generates a new video where the voiceover matches the updated transcription text.

Conclusion

This tool provides an end-to-end workflow for educational content creators, allowing them to rapidly prototype videos and MCQs and then refine them through intuitive text-based editing loops.